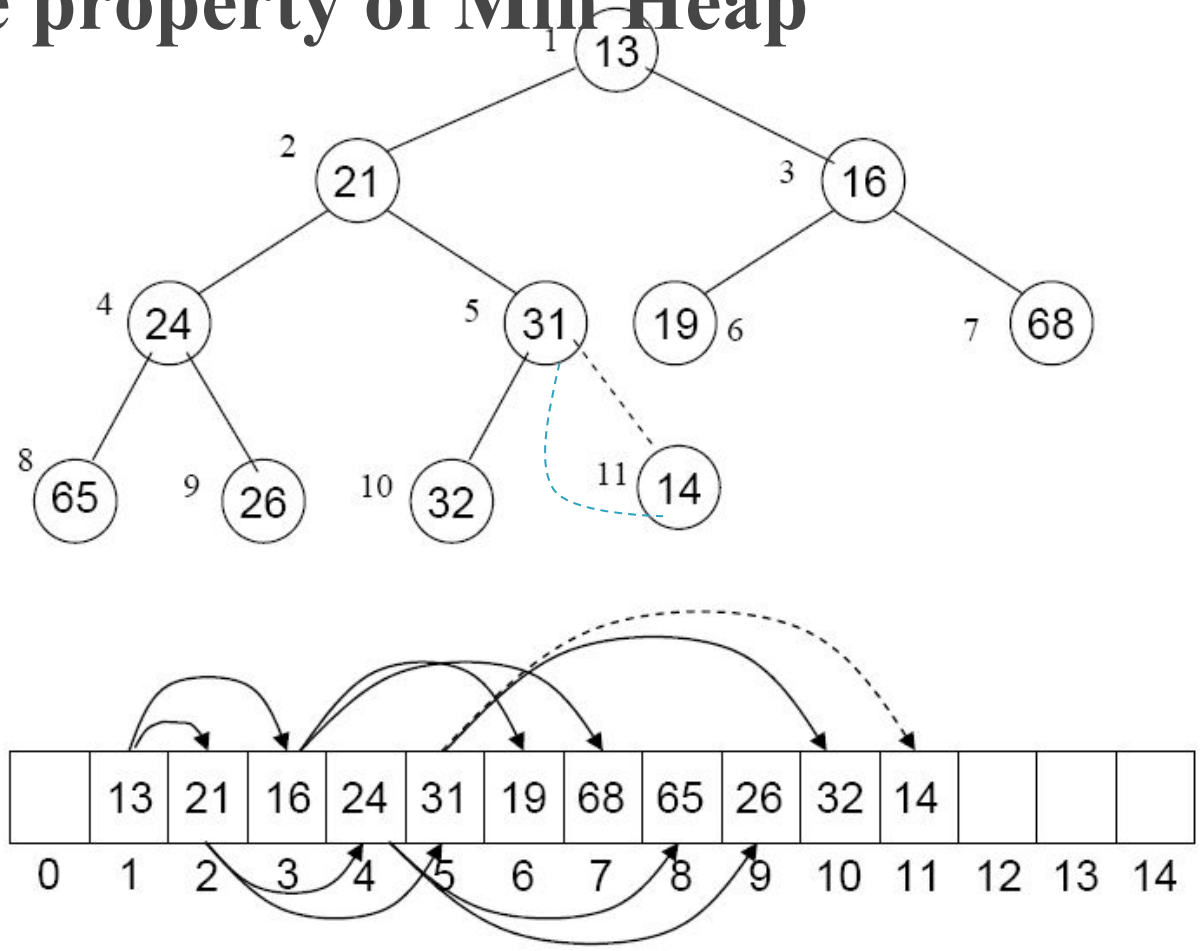
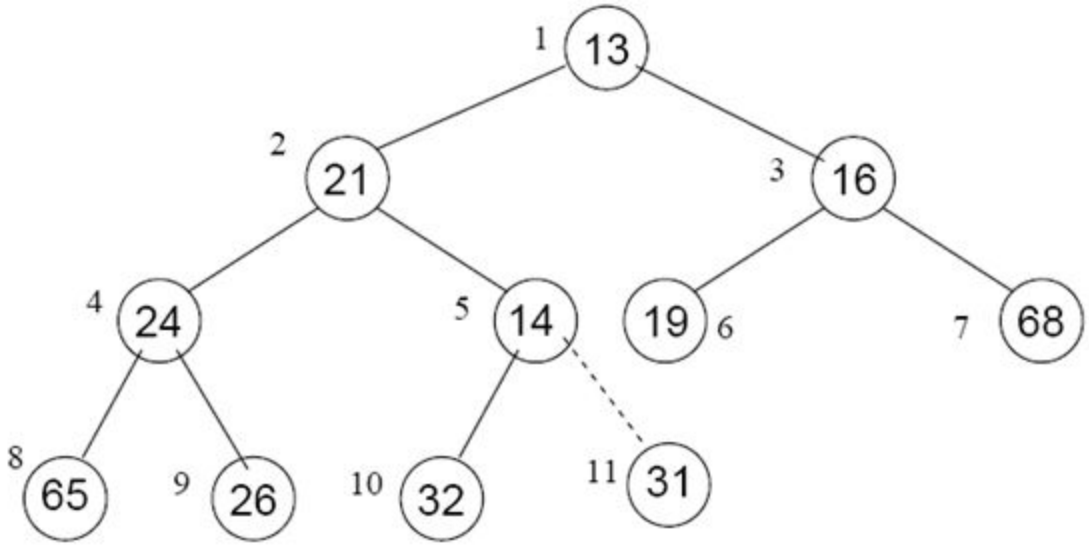
Insertion in a Min Heap

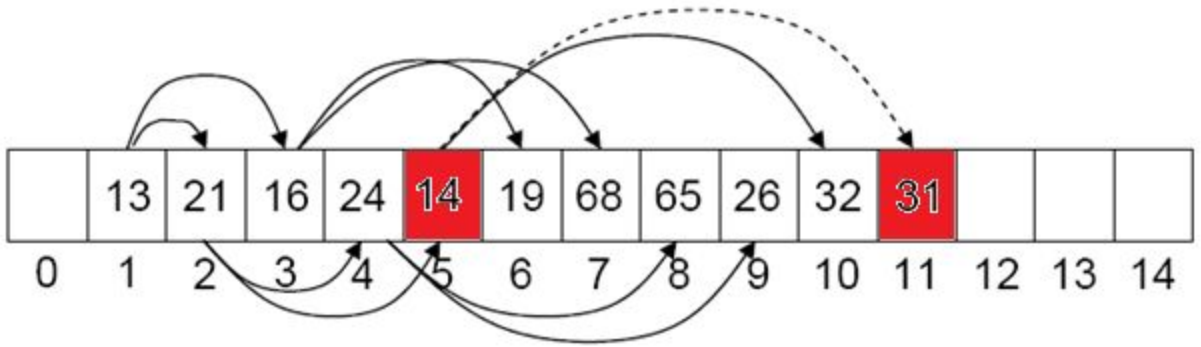
Data Structure

Insert (14)
Position in Complete Binary Tree.
Violate property of Min Heap

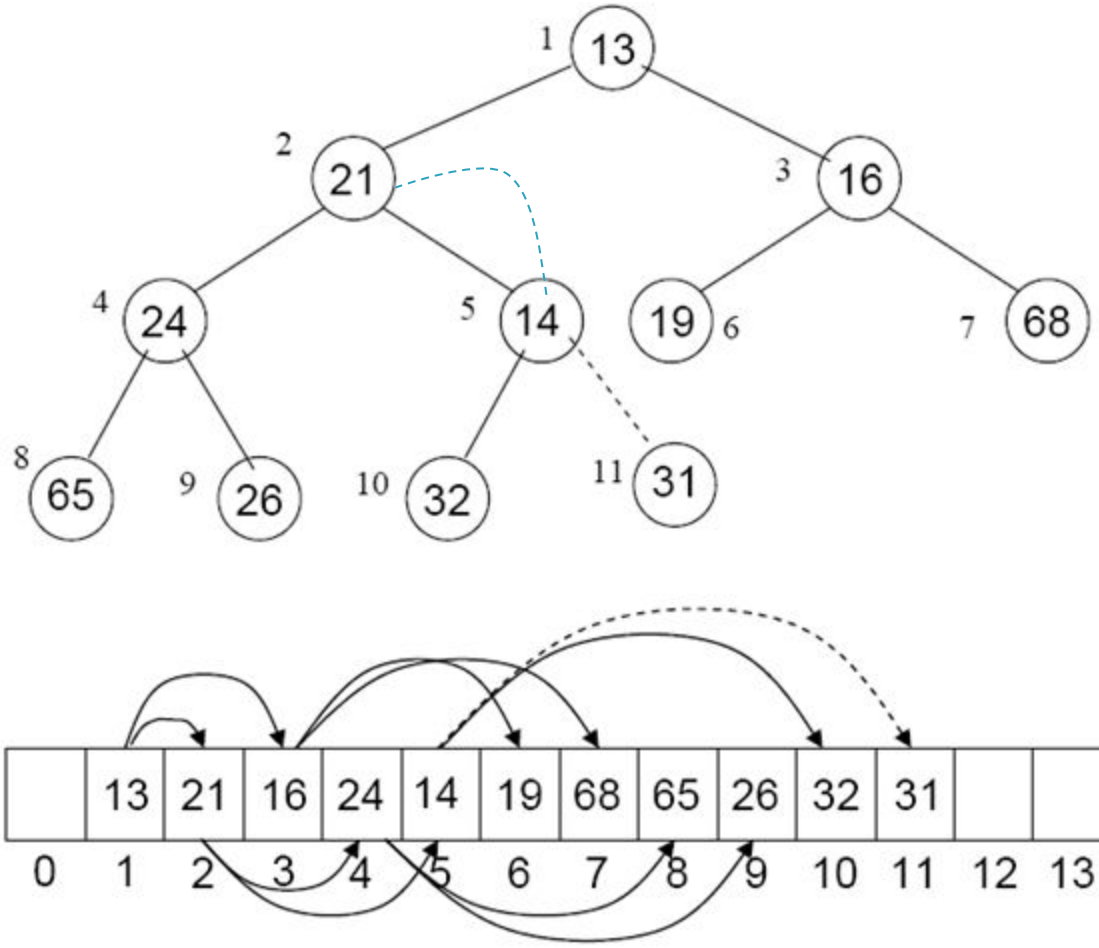


After swapped

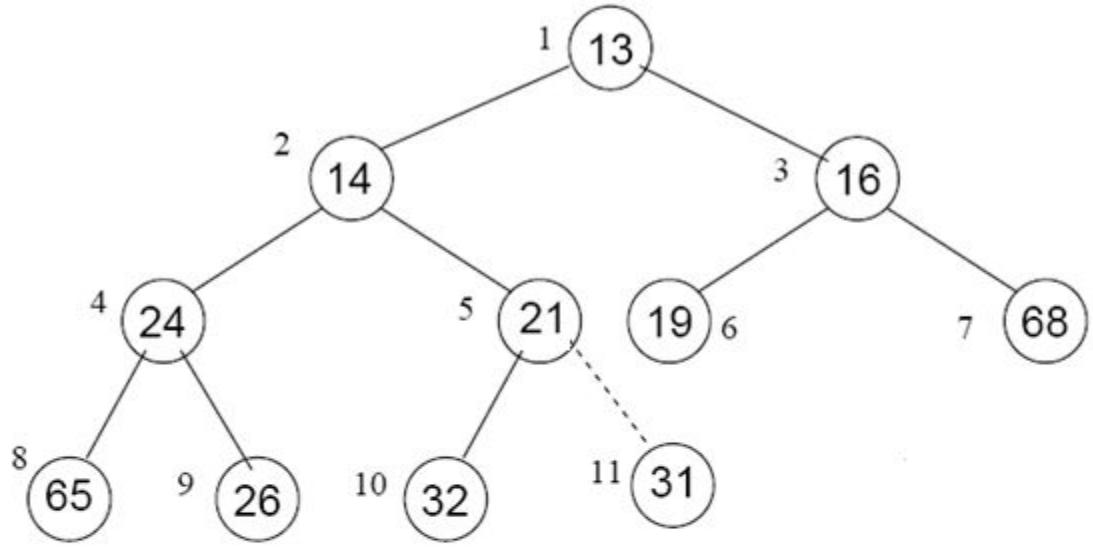


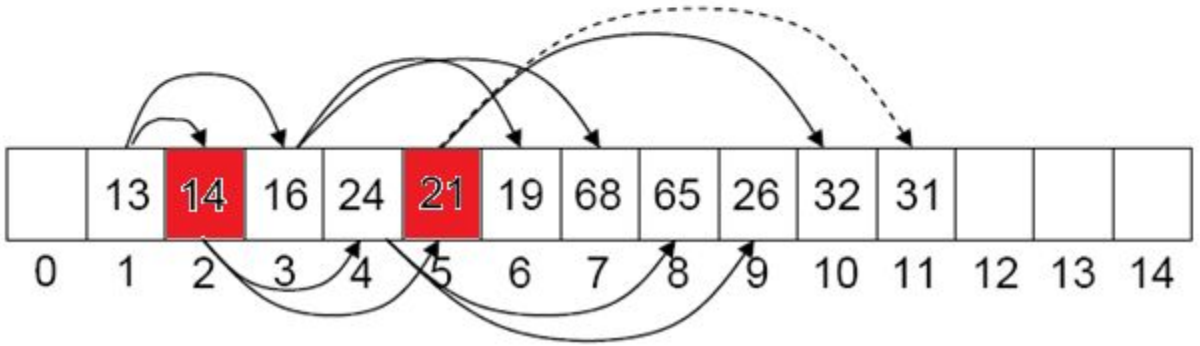


Swap 21 and 14

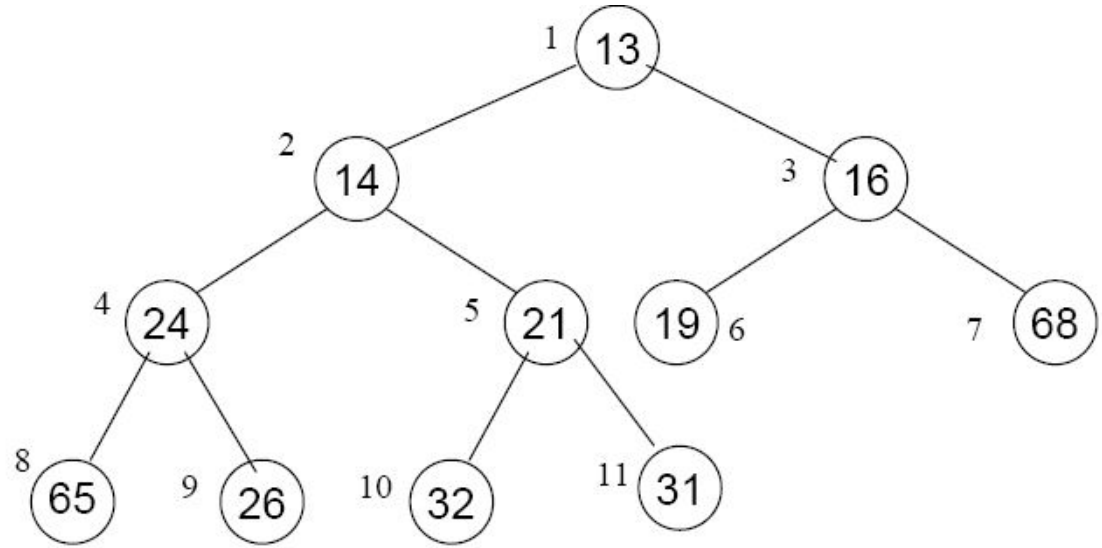


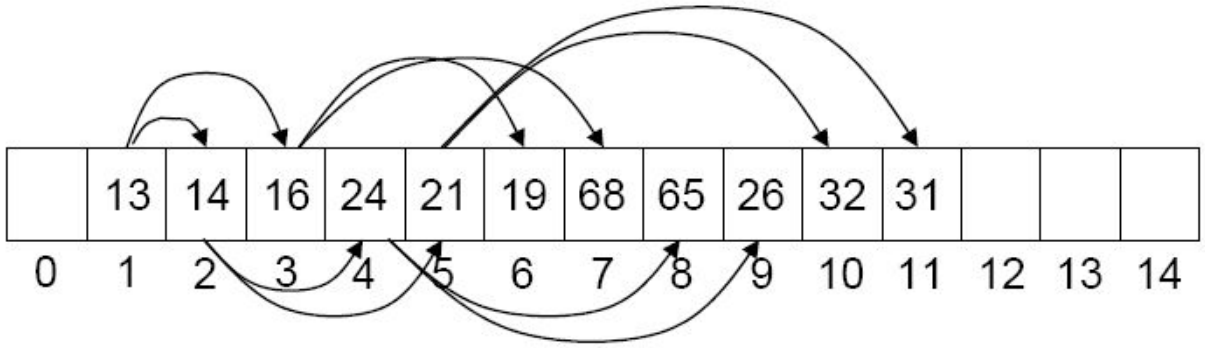
After swapped



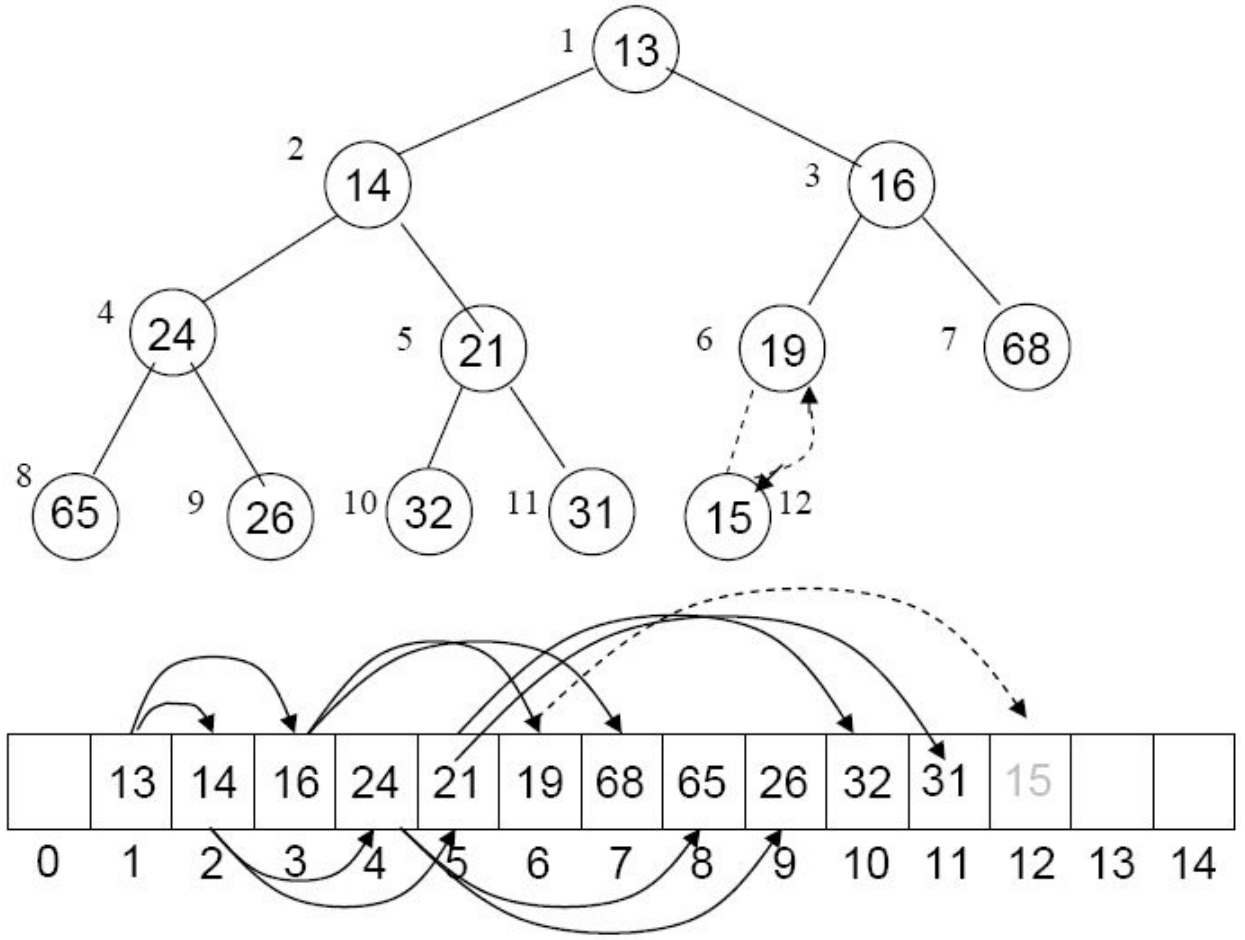


Finally

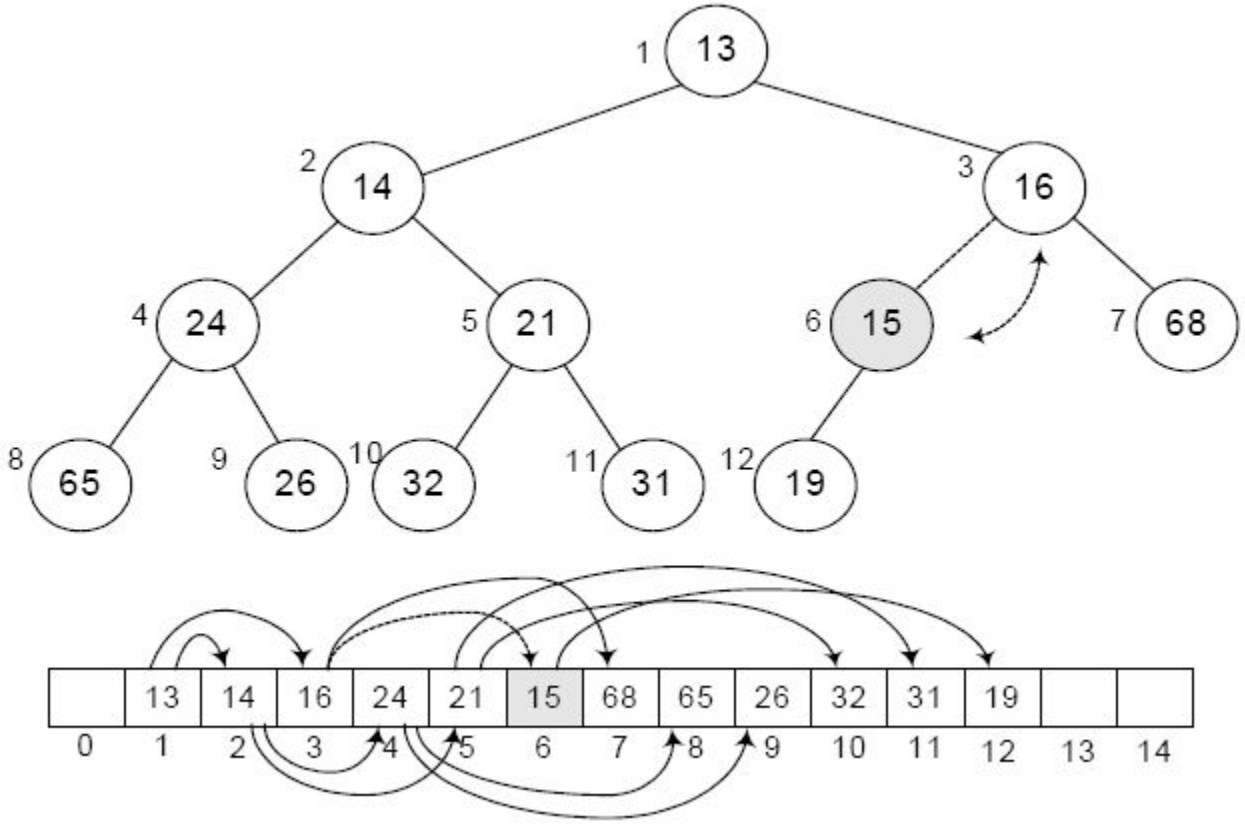

1
13
2
14
3
16
4
24
5
21
19
6
7
68
8
65
9
26
10
32
11
31


13 14 16 24 21 19 68 65 26 32 31
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14

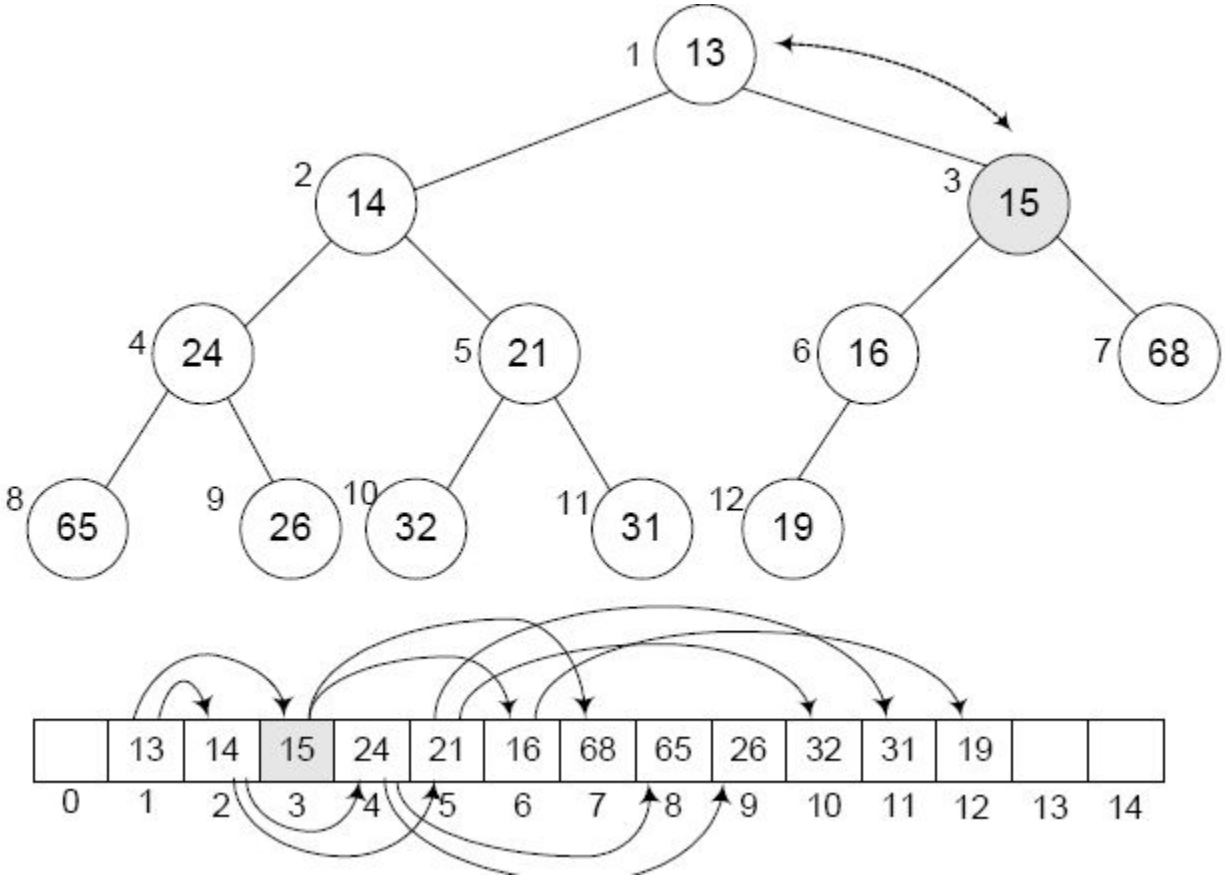
Insert ‘15’ By Swap method



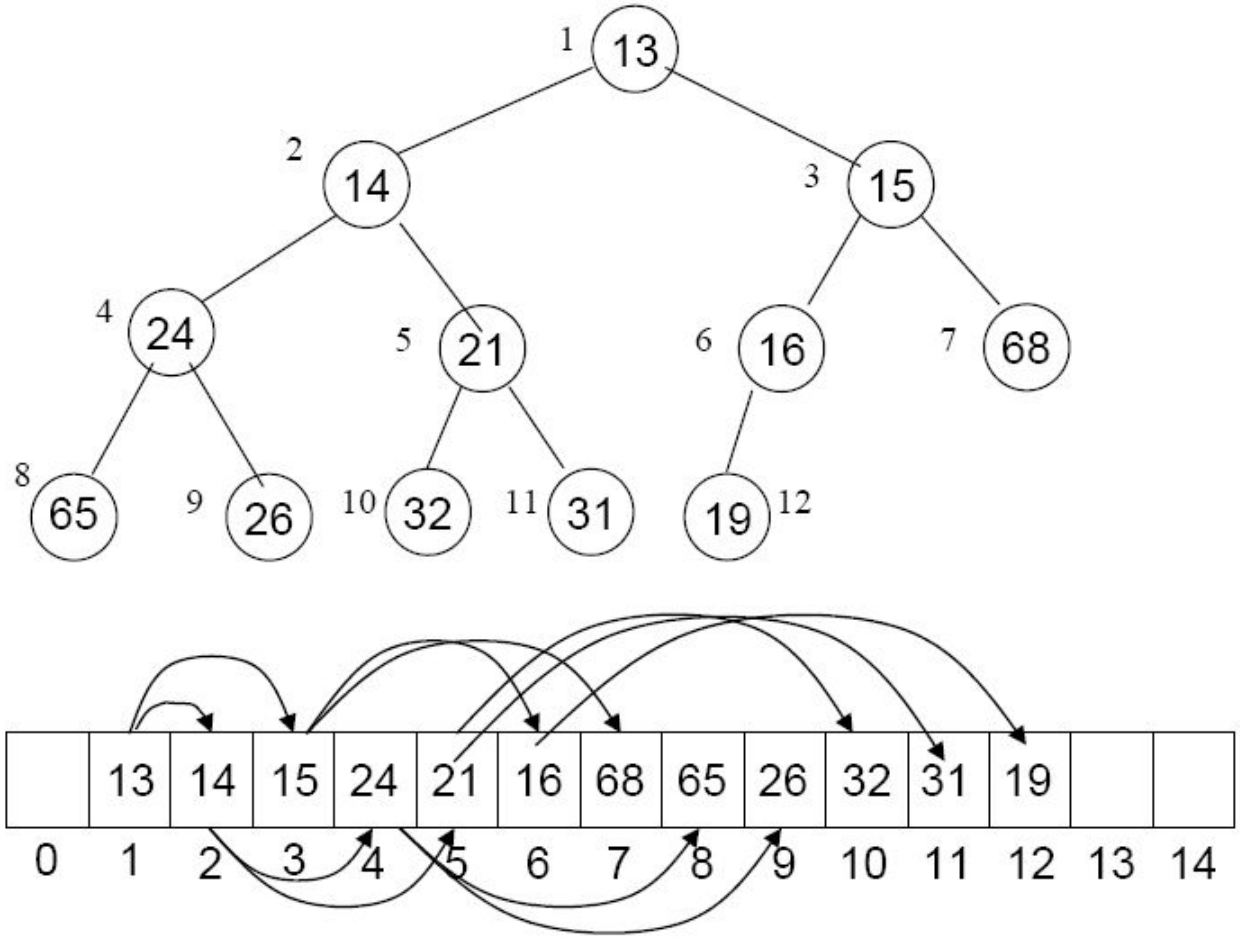
Insert ‘15’ By Swap method



Insert ‘15’ By Swap method



Finally


1
13
2
14
3
15
4
24
5
21
6
16
7
68
8
65
9
26
10
32
11
31
19
12
13 14 15 24 21 16 68 65 26 32 31 19
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14

Min Heap:
build by insert step by step

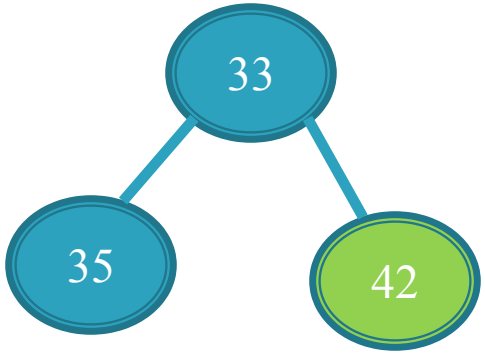
- For Input → 35 33 42 10 14 19 27 44 26 31

Min Heap:
build by insert step by step

- For Input → 35 33 42 10 14 19 27 44 26 31

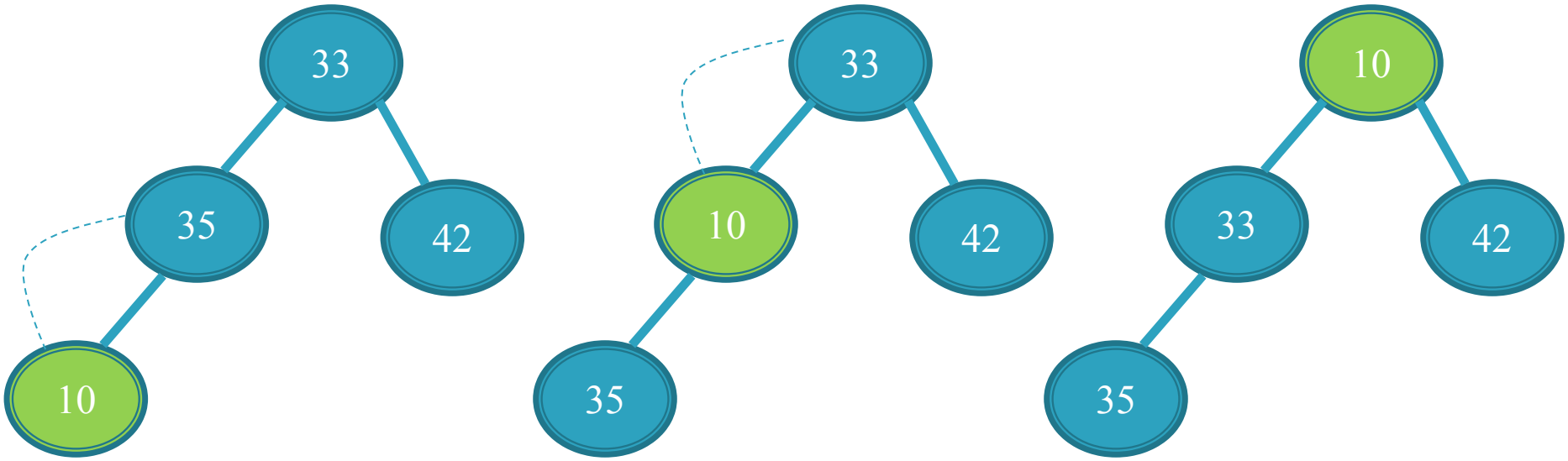
Min Heap:
build by insert step by step

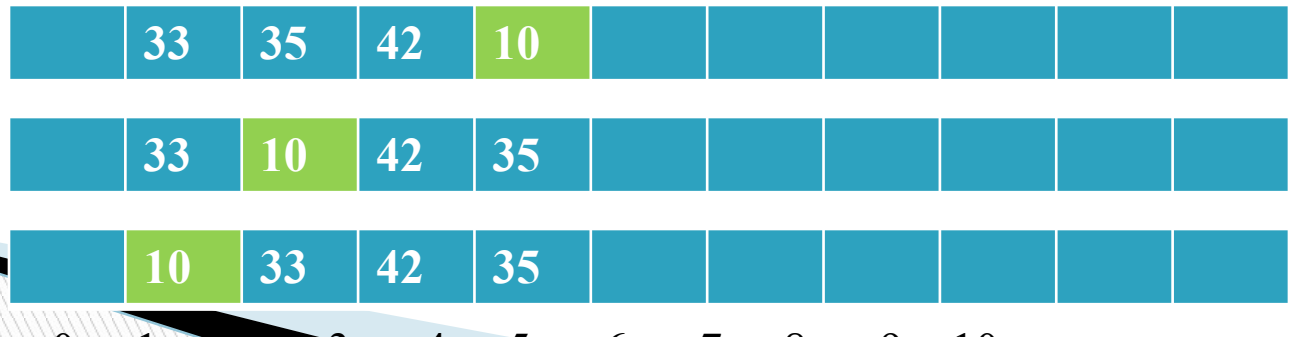
- For Input → 35 33 42 10 14 19 27 44 26 31



Min Heap:
build by insert step by step

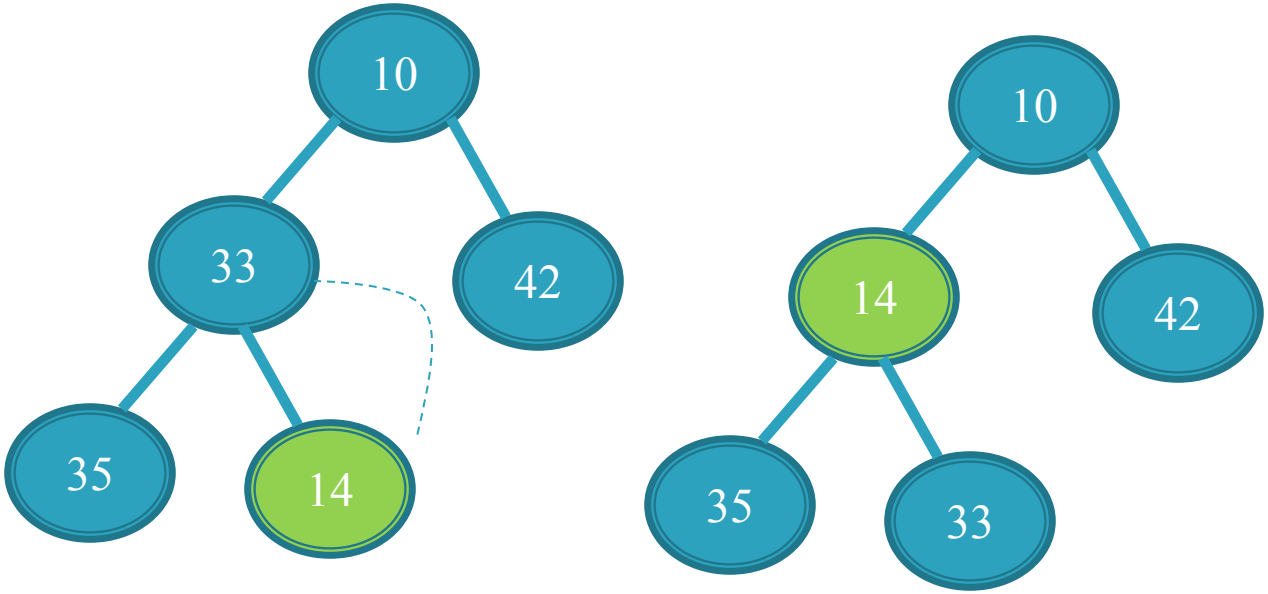
- For Input → 35 33 42 10 14 19 27 44 26 31

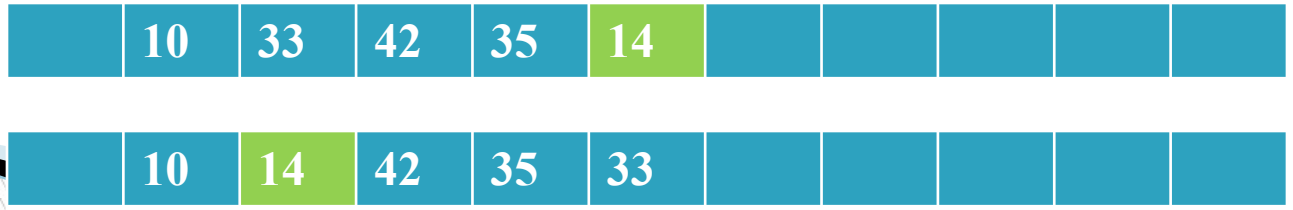




Min Heap:
build by insert step by step

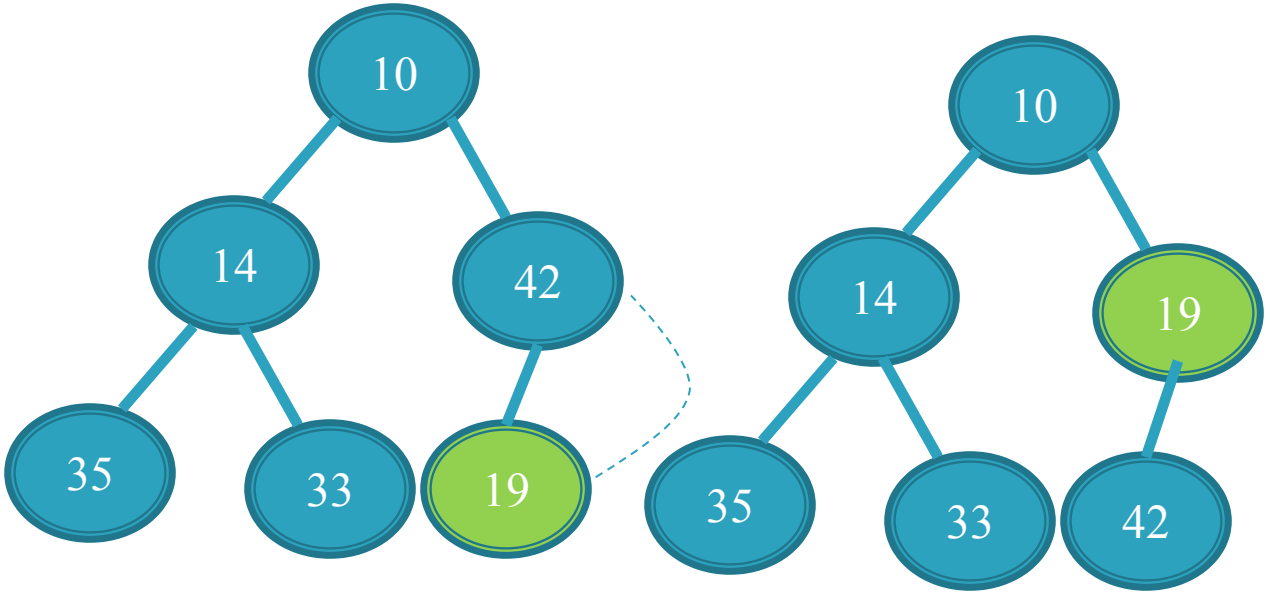
- For Input → 35 33 42 10 14 19 27 44 26 31





Min Heap:
build by insert step by step

- For Input → 35 33 42 10 14 19 27 44 26 31



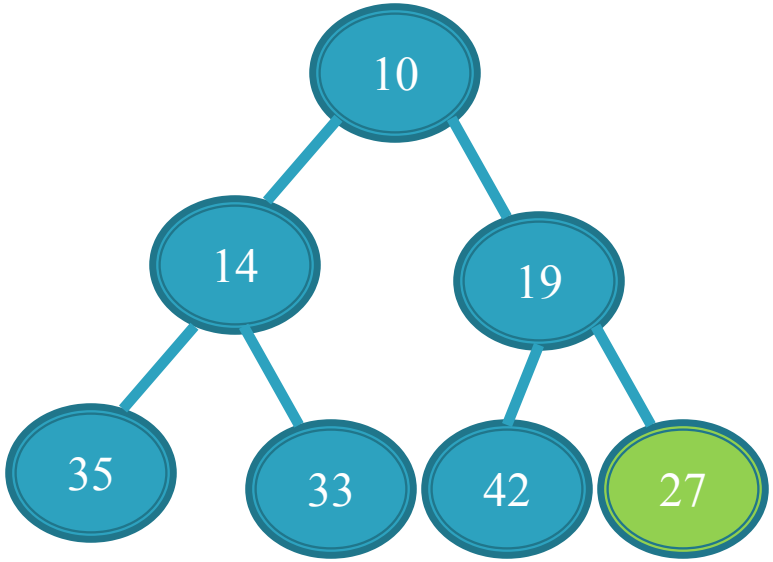
	10	14	42	35	33	19				

	10	14	19	35	33	42				

0 1 2 3 4 5 6 7 8 9 10

Min Heap:
build by insert step by step

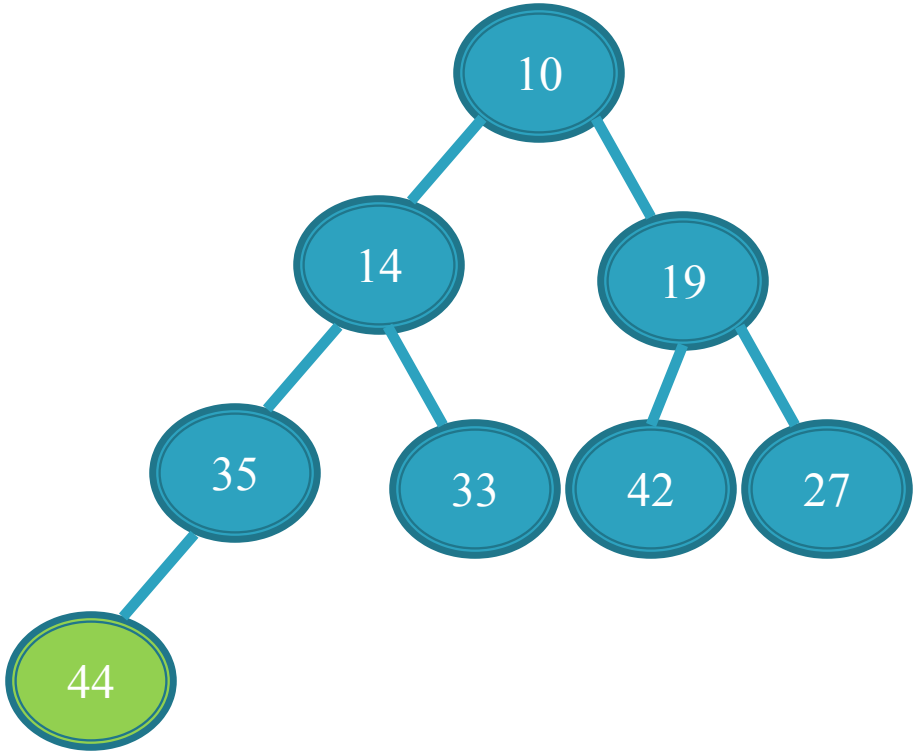
- For Input → 35 33 42 10 14 19 27 44 26 31



	10	14	19	35	33	42	27			
0	1	2	3	4	5	6	7	8	9	10

Min Heap:
build by insert step by step

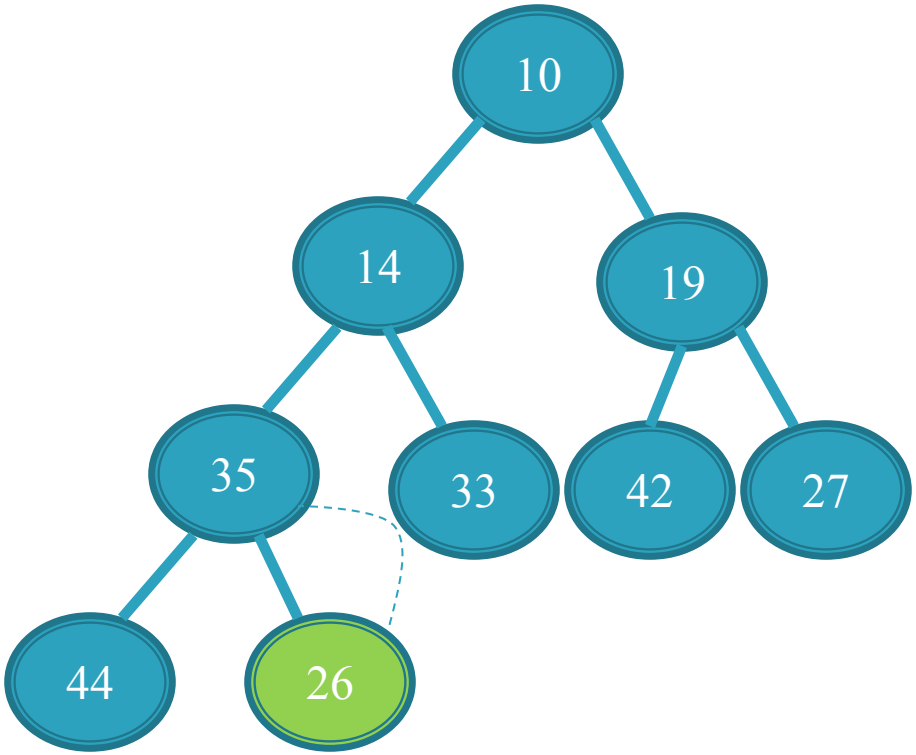
- For Input → 35 33 42 10 14 19 27 44 26 31

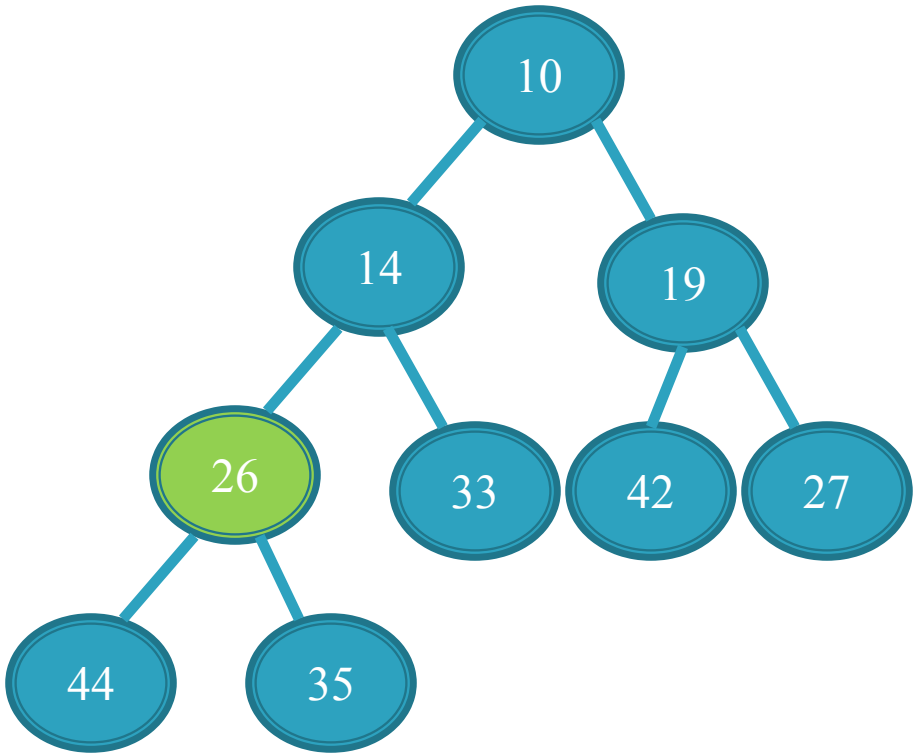


	10	14	19	35	33	42	27	44		
0	1	2	3	4	5	6	7	8	9	10

Min Heap:
build by insert step by step

- For Input → 35 33 42 10 14 19 27 44 26 31

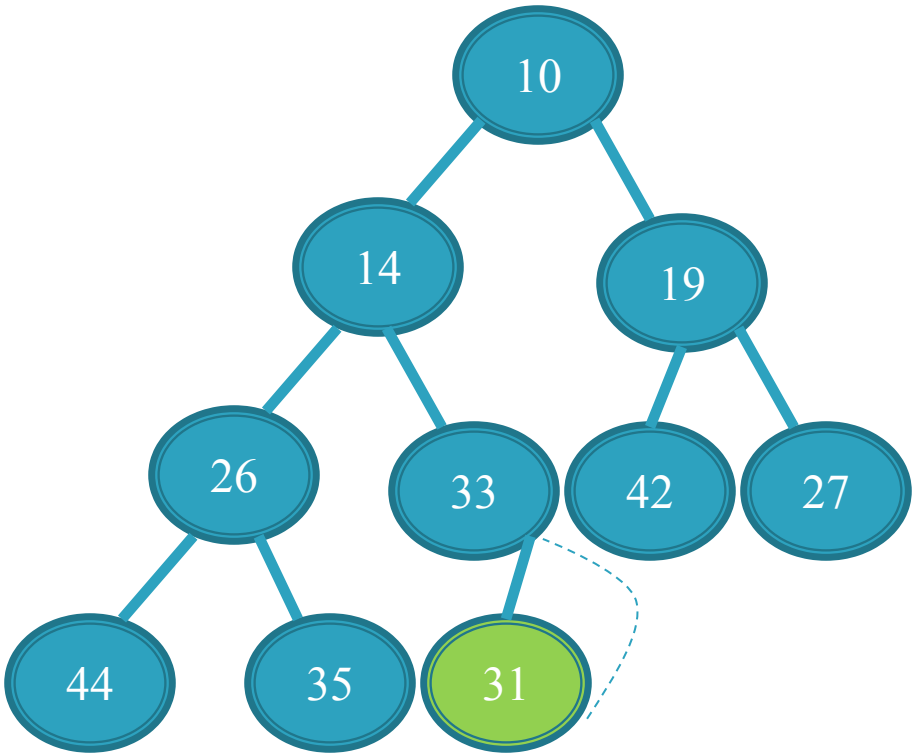


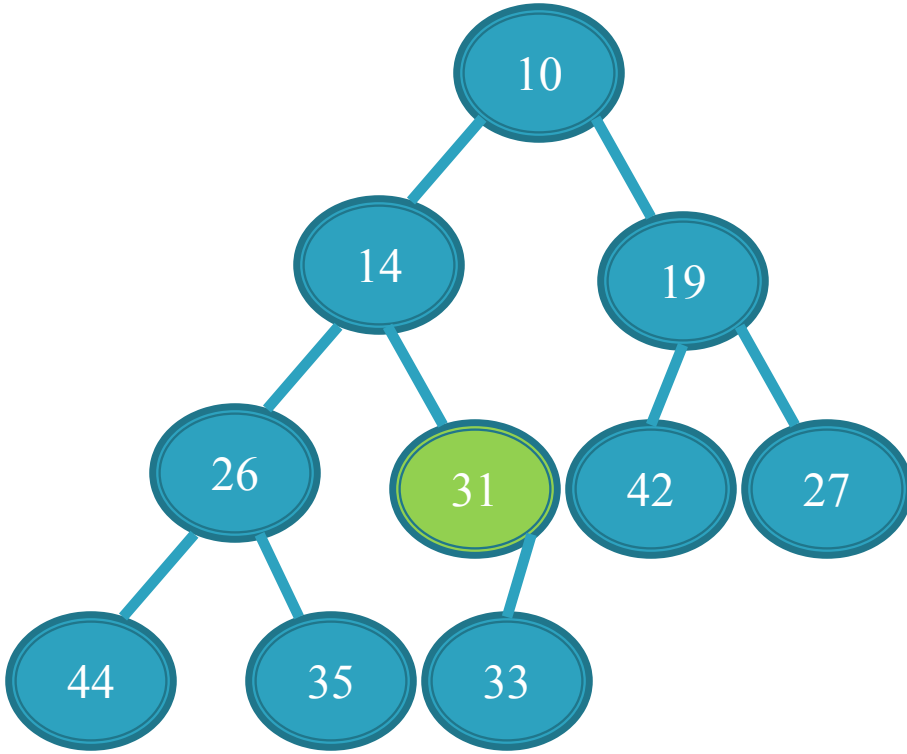


	10	14	19	35	33	42	27	44	26	
	10	14	19	26	33	42	27	44	35	
0	1	2	3	4	5	6	7	8	9	10

Min Heap:
build by insert step by step

- For Input → 35 33 42 10 14 19 27 44 26 31





	10	14	19	26	33	42	27	44	35	31
	10	14	19	26	31	42	27	44	35	33
0	1	2	3	4	5	6	7	8	9	10

Insertion in a Max Heap

Input 35 33 42 10 14 19 27 44 26 31

To view this go to **slide show** mode or press **shift+F5**

Heap Class

```
template <typename eType>
 class Heap
 {public:
 Heap( int capacity =100 );
 void insert( const eType & x );
 void deleteMin( eType & minItem );
 void resize(eType* anArray, int n);
 void Display( );
 const eType & getMin( );
 bool isEmpty( );
 bool isFull( );
 int getSize( );
 void buildHeap(eType* anArray, int n );

 private:
  int currentSize; // Number of elements in heap
 eType* array; // The heap array
 int capacity;
 void percolateDown( int hole ); };
```

Constructor

```
1. template <class eType>
2. Heap<eType>::Heap( int capacity)
3. {
4. array = new eType[capacity + 1];
5. this->capacity=capacity;
6. currentSize=0;
7. }
```

Empty and Full Function

- template <class eType>
- bool Heap<eType>::isEmpty( )
- {
- return currentSize == 0;}

- template <class eType>
- bool Heap<eType>::isFull( )
- {
- return currentSize == capacity;}

Insert Function

```
template <class eType>
void Heap<eType>::insert( const eType & x )
{
if( isFull( ) ) {
resize(array);
}

// Percolate up
int hole = ++currentSize;
for(; hole > 1 && x < array[hole/2 ]; hole /= 2)
 array[ hole ] = array[ hole / 2 ];
array[hole] = x;
}
```

Please note the changing effect of highlighted sign:

For min heap insert function <

For max heap insert function >

Insert Function(swap version)

```
template <class eType>
void Heap<eType>::insert( const eType & x )
{
if( isFull( ) ) {
resize(array);
}

// Percolate up also known Heapify up
int hole = ++currentSize;
array[ hole ]=x;
for(; hole > 1 && array[ hole ] <= array[hole/2 ]; hole /= 2)
 swap (array[ hole ] , array[ hole / 2 ]);
}
```

Please note the changing effect of highlighted sign:

For min heap insert function <

For max heap insert function >

Resize Function

```
template <class eType>
void Heap<eType>::resize(){
eType* oldArray= array;
array= new eType[capacity*2+1];
 for(int i=1; i<=capacity;i++)
 array[i]=oldArray[i];

capacity*=2;
delete []oldArray;

}
```

Delete Min

- eType Heap<eType>::deleteMin( )
- {if(isEmpty( )) {
- cout << "heap is empty." << endl;
- return;
- }

- eType minItem = array[1];
- array[1] = array[currentSize--];
- percolateDown(1);// also known **Heapify down**
- return minItem;
- }

percolateDown

```
template <class eType>
void Heap<eType>::percolateDown( int hole )
{
int child;
eType tmp = array[ hole ];
    for( ; hole * 2 <= currentSize; hole = child ){
    child = hole * 2;
     if( child != currentSize && array[child+1] < array[ child ] )
           child++; // right child is smaller
     if( array[ child ] < tmp )
           array[ hole ] = array[ child ];
     else
            break;}
array[ hole ] = tmp;}
```

Please note the changing effect of highlighted sign:

For min heap function <

For max heap function >

percolateDown (Swaping Version)

```
template <class eType>
void Heap<eType>::percolateDown( int hole )
{
int child;
   for( ; hole * 2 <= currentSize; hole = child ){
   child = hole * 2;
    if( child != currentSize && array[child+1] < array[ child ] )
         child++; // right child is smaller
    if( array[ child ] < array[ hole ] )
         swap(array[ hole ] , array[ child ]);
    else
          break;}}
```

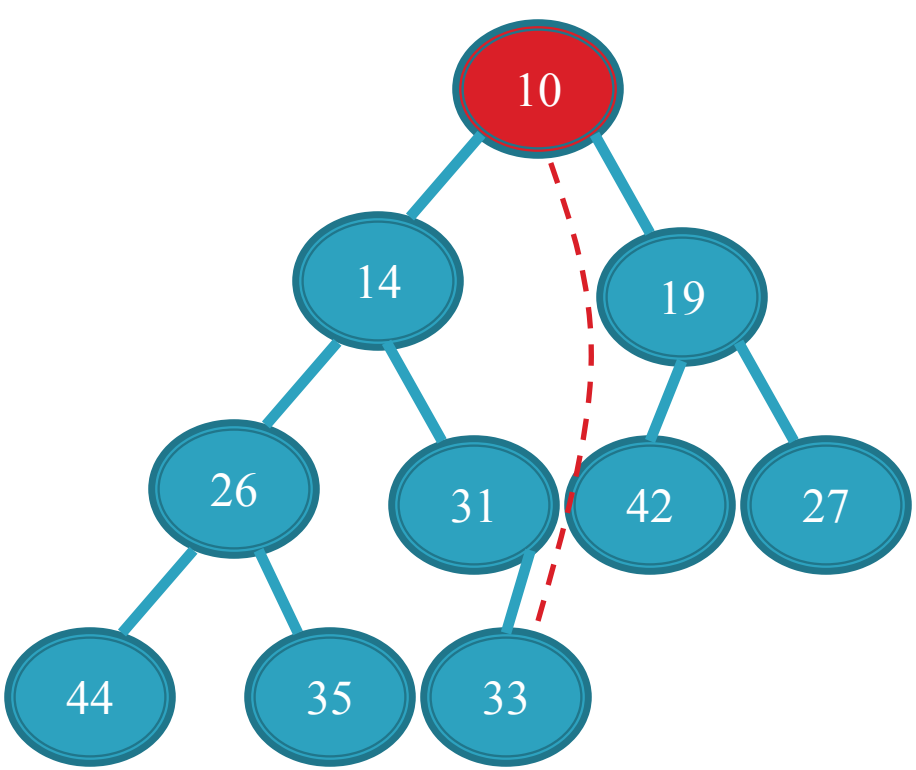
Please note the changing effect of highlighted sign:

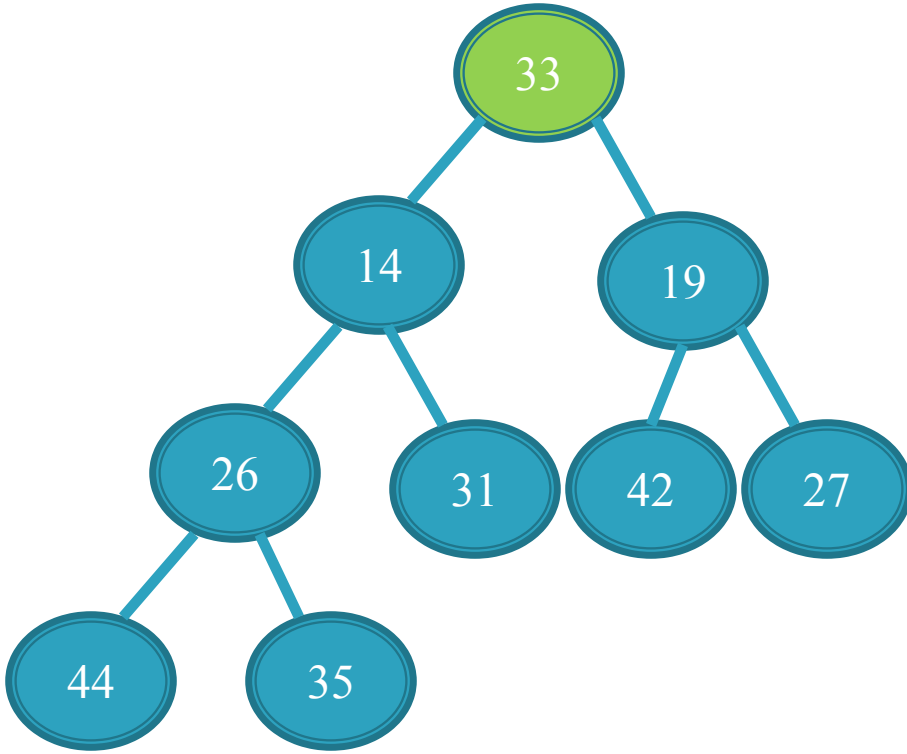
For min heap function <

For max heap function >

Min Heap: remove

- Remove will always remove from index array[1]

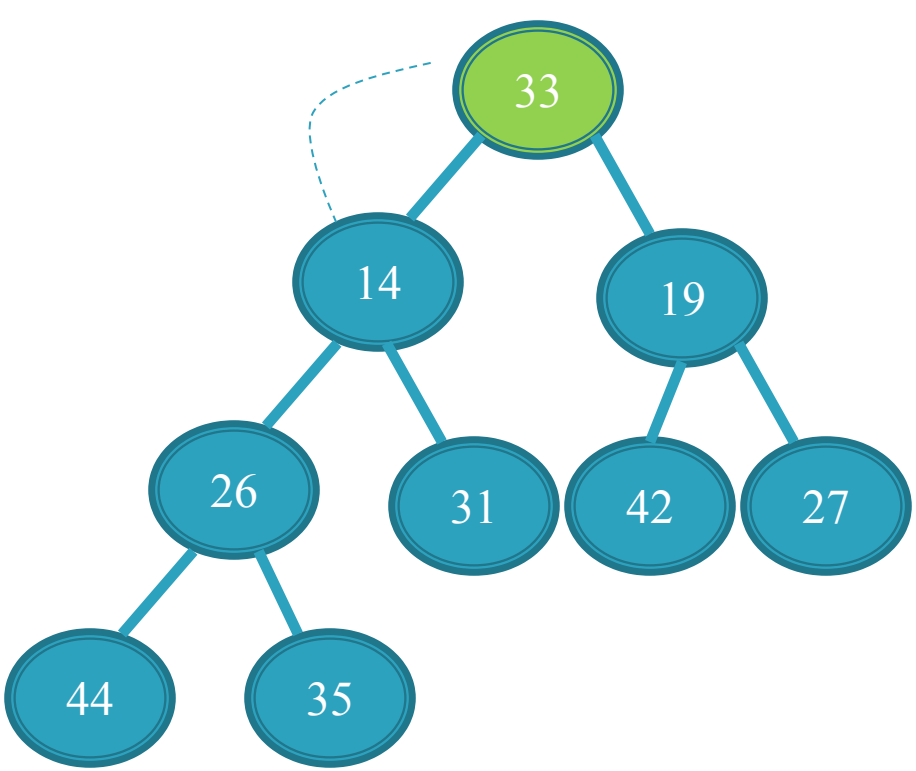


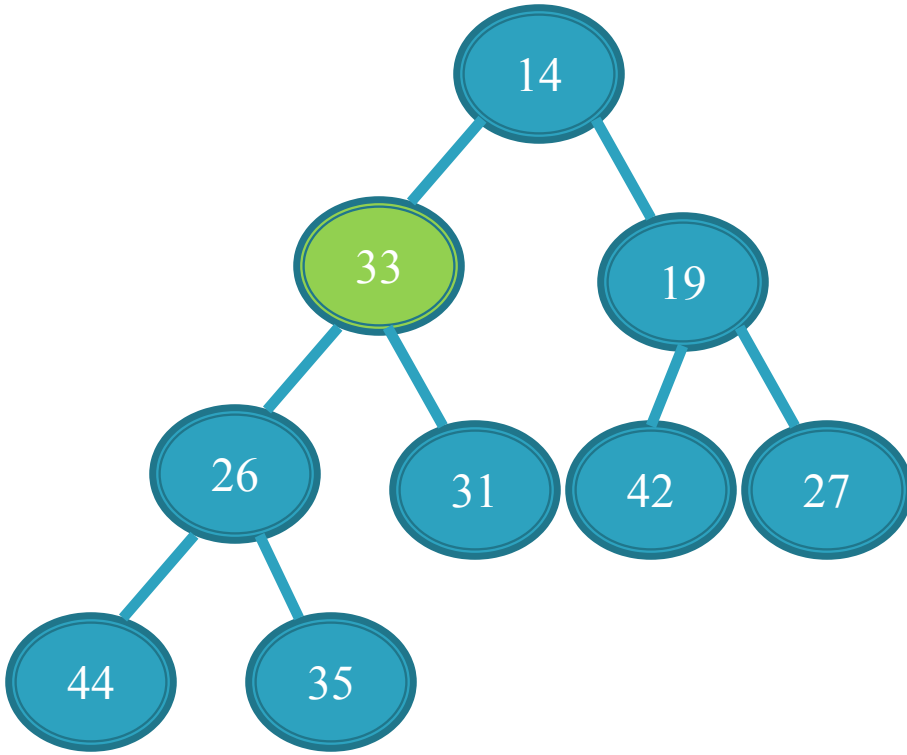


		14	19	26	31	42	27	44	35	33
	33	14	19	26	31	42	27	44	35	
0	1	2	3	4	5	6	7	8	9	10

Min Heap: remove

- Remove will always remove from index array[1]

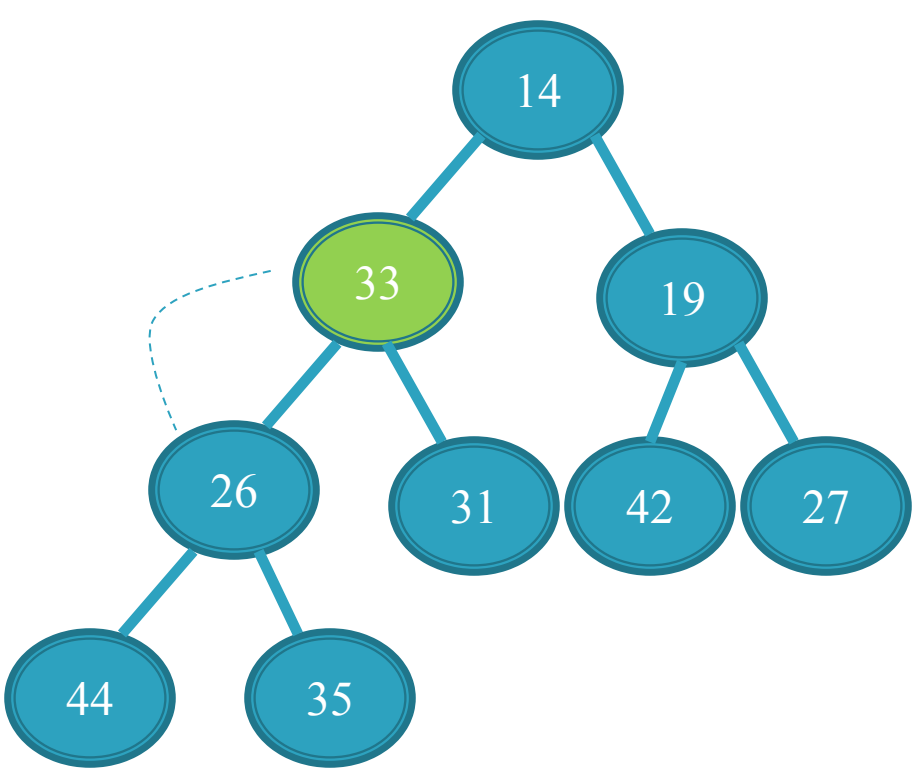


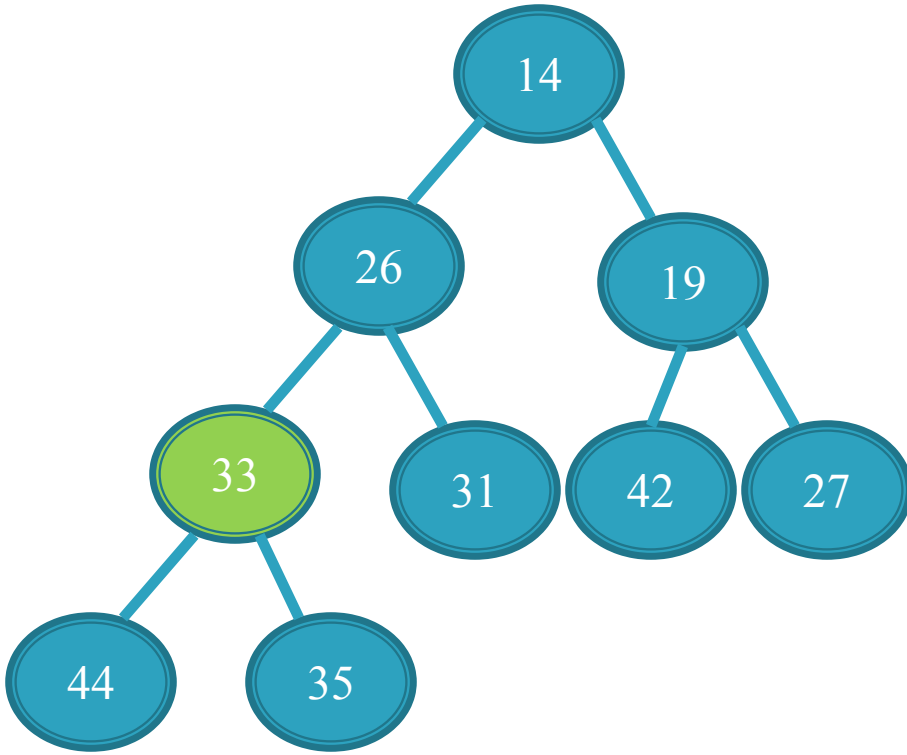


	33	14	19	26	31	42	27	44	35	
	14	33	19	26	31	42	27	44	35	
0	1	2	3	4	5	6	7	8	9	10

Min Heap: remove

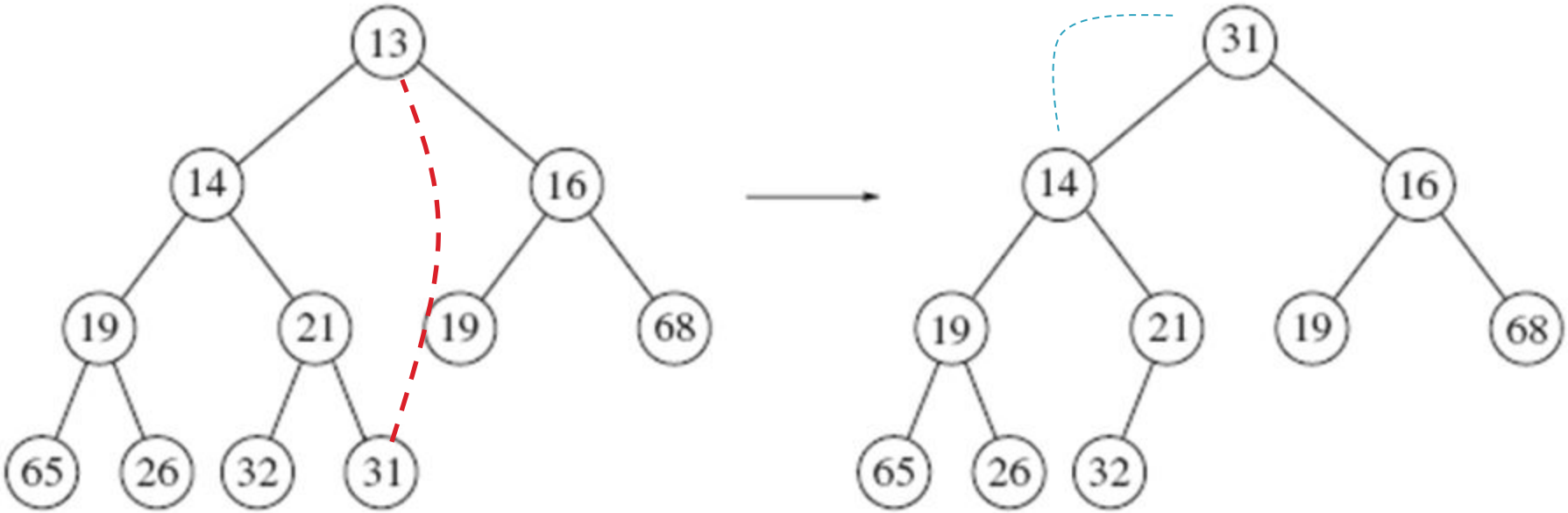
- Remove will always remove from index array[1]





	14	33	19	26	31	42	27	44	35	
	14	26	19	33	31	42	27	44	35	
0	1	2	3	4	5	6	7	8	9	10

Remove From Min Heap



Remove Continuous..

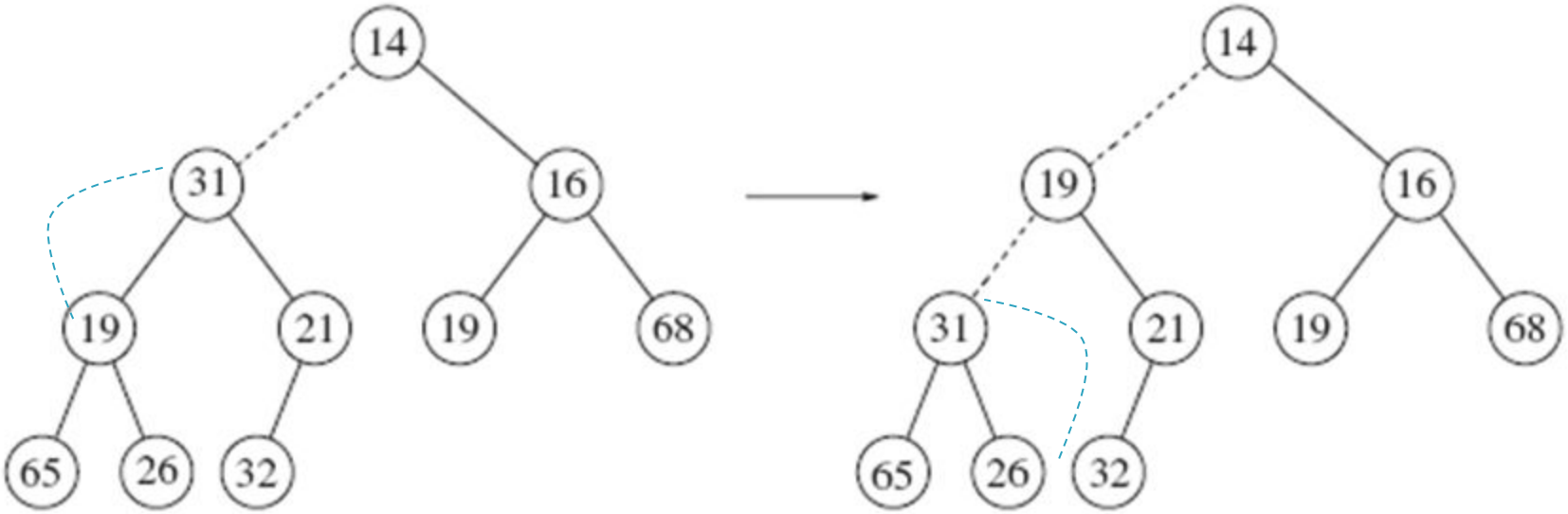
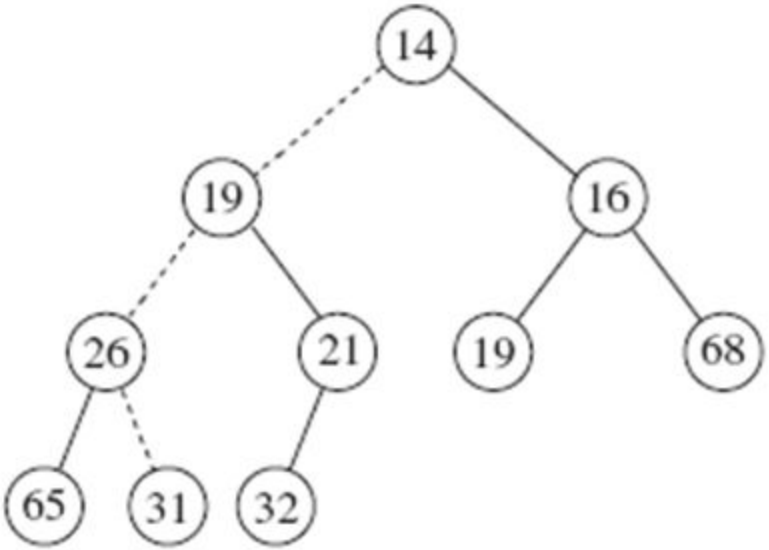
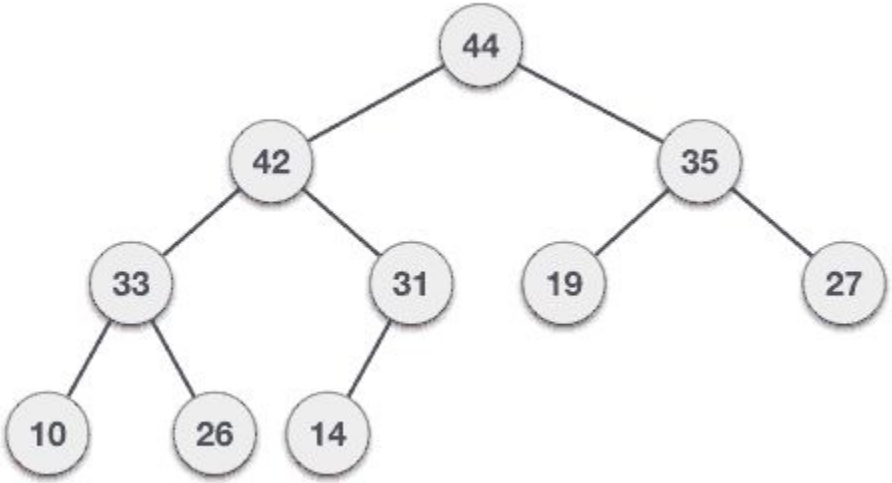


Figure 6.10 Next two steps in `deleteMin`

Remove Continuous..



Remove from Max Heap



To view this go to **slide show** mode or press **shift+F5**